

CS342L – Advanced Database Management Systems Lab

DDL, DML, CONSTRAINTS AND DATABASE MIGRATION



Ghulam Ishaq Khan Institute of Engineering Sciences



Objective

The objective of this session is to migrate an existing benchmark database into PostgreSQL and to practice advanced Data Definition Language (DDL), Data Manipulation Language (DML), and constraint-handling commands.

Instructions

- Open the handout/lab manual in front of a computer in the lab during the session.
- Practice each new command by completing the examples and exercises.
- Turn in the answers for all the exercise problems as your lab report.
- When answering problems, indicate the commands you entered and the output displayed.

Contents

The DVD Rental Benchmark Database	4
Migrating (Restoring) the Benchmark Database	4
Method 1 — Restoring with pg_restore (psql)	4
Method 2 — Restoring with pgAdmin	5
Verifying the Migration.....	5
Categories of SQL Commands.....	5
PostgreSQL CREATE TABLE.....	6
Constraints	6
CREATE TABLE Example — A Small Project-Tracking Schema	6
PostgreSQL ALTER TABLE	7
PostgreSQL DROP TABLE.....	8
PostgreSQL DML: INSERT, UPDATE, DELETE	10
UPDATE with a Subquery	10
DELETE with a Subquery Condition.....	10
INSERT ... SELECT	10
SELECT with a JOIN.....	11
Testing Constraints	11
Exercise	13

The DVD Rental Benchmark Database

For this lab, we will work with the dvdrental sample database — a benchmark database that represents the business processes of a DVD rental store. It contains 15 tables, 1 trigger, 7 views, 8 functions, 1 domain, and 13 sequences, which makes it a realistic dataset for practicing advanced SQL.

The main tables in the dvdrental database are:

- actor – stores actor data (first name and last name).
- film – stores film data such as title, release year, length, and rating.
- film_actor – links films and actors (many-to-many).
- category / film_category – film categories and their link to films.
- store – store locations and their manager.
- inventory – physical copies of films kept at each store.
- rental – rental transactions.
- payment – customer payments.
- staff – store staff.
- customer – registered customers.
- address / city / country – location hierarchy.

Note: Before starting this lab, make sure PostgreSQL and pgAdmin are already installed on your machine, and that you have the dvdrental.tar backup file provided by your instructor (e.g. saved at C:\sampledb\dvdrental.tar).

Migrating (Restoring) the Benchmark Database

Database migration is the process of moving a database's structure and data from a backup file into a live PostgreSQL server. PostgreSQL supports two convenient ways of doing this: the pg_restore command-line tool, and the graphical Restore feature in pgAdmin.

Method 1 — Restoring with pg_restore (psql)

First, launch psql and connect to the PostgreSQL server, then create an empty database to hold the restored data:

```
postgres=# CREATE DATABASE dvdrental;
CREATE DATABASE
```

Code language: SQL (Structured Query Language) (sql)

Exit psql, then navigate to the bin folder of your PostgreSQL installation and run pg_restore, pointing it at the empty database and the backup file:

```
C:\> cd C:\Program Files\PostgreSQL\16\bin
C:\Program Files\PostgreSQL\16\bin> pg_restore -U postgres -d dvdrental
C:\sampledb\dvdrental.tar
```

Code language: Shell Session (shell)

The -U postgres option specifies the login role, and -d dvdrental specifies the target database. Enter the postgres user's password when prompted. The restore normally completes within a few seconds.

Method 2 — Restoring with pgAdmin

Open pgAdmin and connect to your PostgreSQL server. Right-click Databases and choose Create → Database..., name it dvdrental, and click Save. Then right-click the new dvdrental database and choose Restore.... In the dialog, set the Format to Custom or tar, browse to the backup file (e.g. C:\sampledb\dvdrental.tar), and click Restore. pgAdmin will show a progress dialog and confirm Successfully completed once the restore finishes.

Verifying the Migration

Open the Query Tool on the dvdrental database and run the following queries to confirm the restore succeeded:

```
SELECT current_database();

SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;

SELECT COUNT(*) FROM film;
```

Code language: SQL (Structured Query Language) (sql)

The first query should return dvdrental, the second should list all 15 tables, and the third should return the number of films in the catalogue (1000 in the standard dataset).

Categories of SQL Commands

SQL commands are grouped into a few broad categories. This lab focuses on the first two:

- DDL (Data Definition Language) — CREATE, ALTER, DROP: defines and restructures database objects.
- DML (Data Manipulation Language) — INSERT, UPDATE, DELETE: manipulates the data stored in tables.
- DQL (Data Query Language) — SELECT: retrieves data from one or more tables.
- DCL (Data Control Language) — GRANT, REVOKE: controls access to data.
- TCL (Transactional Control) — COMMIT, ROLLBACK, SAVEPOINT: manages transactions.

PostgreSQL CREATE TABLE

A relational database consists of multiple related tables. To create a new table, you use the CREATE TABLE statement:

```
CREATE TABLE [IF NOT EXISTS] table_name (
    column1 datatype(length) column_constraint,
    column2 datatype(length) column_constraint,
    table_constraints
);
```

Code language: SQL (Structured Query Language) (sql)

The IF NOT EXISTS option prevents an error if a table with the same name already exists — PostgreSQL issues a notice instead and skips the creation.

Constraints

PostgreSQL supports the following constraint types:

- NOT NULL – ensures that a column cannot store NULL.
- UNIQUE – ensures the values in a column (or set of columns) are unique across all rows.
- PRIMARY KEY – uniquely identifies each row; a table can have only one primary key, which may span multiple columns (a composite key).
- CHECK – ensures the data satisfies a Boolean expression.
- FOREIGN KEY – ensures a column's values exist in a column of another table; unlike the primary key, a table can have several foreign keys.
- DEFAULT – supplies a value automatically when none is given on INSERT.

CREATE TABLE Example — A Small Project-Tracking Schema

We will build a small project-tracking schema on top of dvdrental that links back to the existing staff table, so you can practice composite keys and multi-table foreign keys. First, the department table:

```
CREATE TABLE department (
    dept_id      SERIAL PRIMARY KEY,
    dept_name    VARCHAR(50) UNIQUE NOT NULL,
    location     VARCHAR(50) NOT NULL DEFAULT 'Head Office'
);
```

Code language: SQL (Structured Query Language) (sql)

Next, the project table, which references department and carries several CHECK constraints, including one that spans two columns:

```
CREATE TABLE project (
    project_id   SERIAL PRIMARY KEY,
    project_name VARCHAR(150) NOT NULL,
    dept_id      INT NOT NULL,
    budget       NUMERIC(12,2) DEFAULT 0,
    start_date   DATE NOT NULL,
    end_date     DATE,
    status       VARCHAR(20) DEFAULT 'PLANNED',
    CHECK (start_date < end_date)
```

```

CONSTRAINT fk_project_department
    FOREIGN KEY (dept_id) REFERENCES department (dept_id),

CONSTRAINT chk_project_budget
    CHECK (budget >= 0),

CONSTRAINT chk_project_dates
    CHECK (end_date IS NULL OR end_date >= start_date),

CONSTRAINT chk_project_status
    CHECK (status IN ('PLANNED', 'ONGOING', 'COMPLETED', 'CANCELLED'))
);

```

Code language: SQL (Structured Query Language) (sql)

Finally, the `project_assignment` table has a composite primary key (`project_id`, `staff_id`) and two foreign keys — one of which points to the existing `staff` table in `dvdrental`:

```

CREATE TABLE project_assignment (
    project_id      INT NOT NULL,
    staff_id        SMALLINT NOT NULL,
    role_title      VARCHAR(50) NOT NULL,
    hours_allocated NUMERIC(5,2) NOT NULL,

    PRIMARY KEY (project_id, staff_id),

    CONSTRAINT fk_assignment_project
        FOREIGN KEY (project_id) REFERENCES project (project_id)
        ON DELETE CASCADE,

    CONSTRAINT fk_assignment_staff
        FOREIGN KEY (staff_id) REFERENCES staff (staff_id),

    CONSTRAINT chk_hours_allocated
        CHECK (hours_allocated > 0 AND hours_allocated <= 60)
);

```

Code language: SQL (Structured Query Language) (sql)

Because the primary key of `project_assignment` spans two columns, it must be declared as a table constraint (as shown above) rather than a column constraint. The `ON DELETE CASCADE` clause means that deleting a project automatically removes its assignments.

PostgreSQL ALTER TABLE

To change the structure of an existing table, you use the `ALTER TABLE` statement. It supports several actions, shown below using the `project` table.

Add a column

```

ALTER TABLE project
ADD COLUMN priority_level SMALLINT DEFAULT 3;

```

Code language: SQL (Structured Query Language) (sql)

Add a CHECK constraint on the new column

```
ALTER TABLE project
ADD CONSTRAINT chk_priority_level
CHECK (priority_level BETWEEN 1 AND 5);
```

Code language: SQL (Structured Query Language) (sql)

Rename a column

```
ALTER TABLE project
RENAME COLUMN priority_level TO priority;
```

Code language: SQL (Structured Query Language) (sql)

Change a column's default value

```
ALTER TABLE project
ALTER COLUMN status
SET DEFAULT 'ONGOING';
```

Code language: SQL (Structured Query Language) (sql)

Drop a constraint

```
ALTER TABLE project
DROP CONSTRAINT chk_priority_level;
```

Code language: SQL (Structured Query Language) (sql)

If you try to insert a row that violates a CHECK constraint, PostgreSQL rejects it and reports which constraint failed:

```
INSERT INTO project (project_id, project_name, dept_id, budget, start_date, end_date, status)
VALUES (5, 'Website Revamp', 2, 50000.00, '2026-09-01', '2026-08-01', 'PLANNED');
```

Code language: SQL (Structured Query Language) (sql)

```
ERROR:  new row for relation "project" violates check constraint "chk_project_dates"
DETAIL:  Failing row contains (5, Website Revamp, 2, 50000.00, 2026-09-01, 2026-08-01,
PLANNED, ...).
```

PostgreSQL DROP TABLE

To remove a table, use DROP TABLE. If other objects (such as a foreign key in another table) depend on it, PostgreSQL refuses the drop unless you add CASCADE:

```
DROP TABLE IF EXISTS project_assignment;
DROP TABLE department CASCADE;
```

Code language: SQL (Structured Query Language) (sql)

The CASCADE option removes the table together with any dependent objects (such as foreign-key constraints in other tables that reference it); the default RESTRICT behaviour blocks the drop instead.

Because the rest of this lab continues to work with department, project, and project_assignment, recreate them (with the same definitions used earlier) before moving on:

```
CREATE TABLE department (
    dept_id      SERIAL PRIMARY KEY,
    dept_name    VARCHAR(50) UNIQUE NOT NULL,
    location     VARCHAR(50) NOT NULL DEFAULT 'Head Office'
);

CREATE TABLE project (
    project_id   SERIAL PRIMARY KEY,
    project_name VARCHAR(150) NOT NULL,
    dept_id      INT NOT NULL,
    budget       NUMERIC(12,2) DEFAULT 0,
    start_date   DATE NOT NULL,
    end_date     DATE,
    status       VARCHAR(20) DEFAULT 'PLANNED',
    CONSTRAINT fk_project_department
        FOREIGN KEY (dept_id) REFERENCES department (dept_id),
    CONSTRAINT chk_project_budget
        CHECK (budget >= 0),
    CONSTRAINT chk_project_dates
        CHECK (end_date IS NULL OR end_date >= start_date),
    CONSTRAINT chk_project_status
        CHECK (status IN ('PLANNED', 'ONGOING', 'COMPLETED', 'CANCELLED'))
);

CREATE TABLE project_assignment (
    project_id   INT NOT NULL,
    staff_id     SMALLINT NOT NULL,
    role_title   VARCHAR(50) NOT NULL,
    hours_allocated NUMERIC(5,2) NOT NULL,
    PRIMARY KEY (project_id, staff_id),
    CONSTRAINT fk_assignment_project
        FOREIGN KEY (project_id) REFERENCES project (project_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_assignment_staff
        FOREIGN KEY (staff_id) REFERENCES staff (staff_id),
    CONSTRAINT chk_hours_allocated
        CHECK (hours_allocated > 0 AND hours_allocated <= 60)
);
```

Code language: SQL (Structured Query Language) (sql)

PostgreSQL DML: INSERT, UPDATE, DELETE

With the schema in place, populate department and project, then use INSERT ... RETURNING to see what was written back immediately:

```
INSERT INTO department (dept_name, location)
VALUES
    ('Engineering', 'Building A'),
    ('Marketing', 'Building B')
RETURNING dept_id, dept_name;
```

Code language: SQL (Structured Query Language) (sql)

Insert a new project referencing the department that was just created, and return the generated project_id:

```
INSERT INTO project (project_name, dept_id, budget, start_date, status)
VALUES
    ('Customer Portal Redesign', 1, 85000.00, '2026-09-15', 'ONGOING')
RETURNING project_id, project_name, status;
```

Code language: SQL (Structured Query Language) (sql)

UPDATE with a Subquery

A more realistic UPDATE often relies on a subquery rather than a hard-coded value. The following statement raises the budget of every project belonging to the Engineering department by 10%:

```
UPDATE project
SET budget = budget * 1.10
WHERE dept_id = (
    SELECT dept_id FROM department WHERE dept_name = 'Engineering'
)
RETURNING project_id, project_name, budget;
```

Code language: SQL (Structured Query Language) (sql)

DELETE with a Subquery Condition

DELETE statements can also filter using a subquery. The following removes any assignment for projects that have already been marked COMPLETED:

```
DELETE FROM project_assignment
WHERE project_id IN (
    SELECT project_id FROM project WHERE status = 'COMPLETED'
)
RETURNING project_id, staff_id;
```

Code language: SQL (Structured Query Language) (sql)

INSERT ... SELECT

INSERT ... SELECT populates a table using the result of a query against another table — useful for migrating or deriving data. The following assigns every member of staff who works at store 1 to project 1, with a default role and 10 allocated hours:

```
INSERT INTO project_assignment (project_id, staff_id, role_title, hours_allocated)
SELECT
    1,
    staff_id,
    'Contributor',
    10
FROM staff
WHERE store_id = 1
RETURNING project_id, staff_id, role_title;
```

Code language: SQL (Structured Query Language) (sql)

SELECT with a JOIN

Because project_assignment references both project and staff, a JOIN is the natural way to read it back in a meaningful form:

```
SELECT
    p.project_name,
    s.first_name || ' ' || s.last_name AS staff_member,
    pa.role_title,
    pa.hours_allocated
FROM project_assignment pa
JOIN project p ON p.project_id = pa.project_id
JOIN staff s ON s.staff_id = pa.staff_id
ORDER BY p.project_name, pa.hours_allocated DESC;
```

Code language: SQL (Structured Query Language) (sql)

Testing Constraints

Constraints exist to reject invalid data before it is written. Try the following statement, which violates the composite primary key of project_assignment by re-assigning the same staff member to the same project:

```
INSERT INTO project_assignment (project_id, staff_id, role_title, hours_allocated)
VALUES (1, 1, 'Duplicate Attempt', 5);
```

Code language: SQL (Structured Query Language) (sql)

```
ERROR: duplicate key value violates unique constraint "project_assignment_pkey"
DETAIL: Key (project_id, staff_id)=(1, 1) already exists.
```

This statement violates the CHECK constraint on hours_allocated, since 0 is outside the allowed range:

```
INSERT INTO project_assignment (project_id, staff_id, role_title, hours_allocated)
VALUES (1, 2, 'Reviewer', 0);
```

Code language: SQL (Structured Query Language) (sql)

```
ERROR: new row for relation "project_assignment" violates check constraint
"chk_hours_allocated"
DETAIL: Failing row contains (1, 2, Reviewer, 0.00).
```

This statement violates the foreign key on dept_id, since department 99 does not exist:

```
INSERT INTO project (project_name, dept_id, start_date)
VALUES ('Ghost Project', 99, '2026-10-01');
```

Code language: SQL (Structured Query Language) (sql)

```
ERROR: insert or update on table "project" violates foreign key constraint
"fk_project_department"
DETAIL: Key (dept_id)=(99) is not present in table "department".
```

Exercise

These exercises use the dvdrental database you migrated at the start of this lab — in particular the film, actor, film_actor, category, film_category, customer, staff, rental, payment, inventory, store, address, city, country, and language tables. Every query below runs directly against the existing data, so no fabricated IDs are needed; where a specific value such as a customer_id is required, the one shown is present in the standard dvdrental dataset. Run each query and record the output as your lab report.

1. Display all the data from the rental table for customer_id = 1.
2. Write a SQL query to find the total amount paid (SUM of payment.amount) per customer, grouped by customer_id, ordered from highest to lowest total (use GROUP BY and SUM).
3. Show the film title, category name, and language name for every film in the catalogue, using JOINS with film_category, category, and language.
4. Write a query that lists only the films whose rental_rate is above the average rental_rate of all films (use a subquery).
5. Add a CHECK constraint to the film table ensuring that rental_duration is always greater than 0.
6. Add a new column total_rentals to the customer table with a DEFAULT of 0. Then use an UPDATE statement with a correlated subquery to set total_rentals to each customer's actual number of rentals, and use RETURNING to display the updated rows.
7. Rename the film_category table to film_genre, then rename it back to film_category.